Data Structures

GapBuffer

The primary data structure used in my text editor is the **GapBuffer**, a common choice for text editors because it allows for near **O(1) insertion** at the cursor position. One of the most well-known editors that uses a GapBuffer is GNU Emacs.

A GapBuffer is particularly useful because inserting text into a standard array can be expensive when the cursor is not at the end of the line. In a normal array, every character after the insertion point must be shifted to make room for the new character. A GapBuffer avoids this by maintaining a **gap** (or empty space) at the cursor location.

Example:

```
_ _ _ _ _ _ _ _ _ _  <- Empty GapBuffer of size 10.
                        The entire array is initially the gap.

H e r e _ _ _ _ _ _  <- Characters are inserted.
                        The gap shrinks as space is used.

H e _ _ _ _ _ _ r e  <- The cursor moves two positions left.
                        The characters 'r' and 'e' are shifted right,
                        placing the gap at the new cursor position.
```

The key idea is that the gap is always located at the cursor. As a result, inserting characters requires no shifting and can be performed efficiently. Since my editor only allows cursor movement through the arrow keys, I did not need to handle large cursor jumps.

I implemented the GapBuffer as a `GapBuffer` class. Internally, it stores text in a `char[] line` array. Empty positions are represented by the null character (`'\u0000'`), which is the default value for uninitialized elements in a character array.

The core methods are:

- `left()` – Moves the cursor left.
- `right()` – Moves the cursor right.
- `insert()` – Inserts a character at the cursor.
- `remove()` – Deletes a character.
- `resize()` – Expands the array when the gap is exhausted.

The implementation primarily consists of array manipulation to maintain the gap and keep characters in the correct positions.

Most GapBuffer implementations refer to the cursor position as **GapStart**. I chose to name the variable `cursor` because it more clearly represents its purpose. I also maintain a variable called `max_cursor`, which represents the position immediately after the last character in the line (as if no gap existed). In other words, it defines the furthest position the cursor can move to, which matches the behavior of most text editors.

Some editors, such as Emacs, store the entire document as a single GapBuffer and determine line breaks by reading newline (`'\n'`) characters. Instead, I chose to represent the document as an `ArrayList<GapBuffer>`, where each GapBuffer corresponds to a single line. This approach simplifies vertical cursor movement and line-based operations.

Stack and Node

To implement undo functionality, I used a stack to maintain an edit history. Each edit is stored as a `Node` object and pushed onto the undo stack.

The `Node` class stores all information necessary to reverse an edit. It contains a single constructor that initializes the following fields:

- `char type`
- `String s`
- `int height`
- `int start`

The `type` field identifies the kind of edit being undone, such as:

- Text insertion
- Text deletion
- New line creation
- Line deletion

The `String s` field stores any text involved in the operation. For example:

- When text is inserted or deleted, `s` stores the affected text.
- When a new line is created, `s` stores the text moved from the original line to the new line.
- When a line is deleted, `s` stores the text that was merged into the previous line.

The `height` field stores the line number required to perform the undo. For both line creation and line deletion, I use the previous line number because that is where the cursor ultimately resides after the operation.

The `start` field stores the cursor position needed to correctly restore the edit.

For redo functionality, I created a separate class called `RedoStack`. Despite its name, it is not actually a stack. Instead, it acts as a container for the information needed to perform the most recent redo operation. Unlike `Node`, it contains multiple constructors because different redo actions require different sets of information.

Undo and Redo System

Storing every individual character insertion or deletion as a separate undo operation would be inefficient, since each character would require its own `Node` object. Most text editors therefore group related edits into larger undo actions.

My editor uses a simple grouping strategy. Two `StringBuilder` objects are maintained:

- One for recently entered text.
- One for recently deleted text.

Text Insertion

Characters typed consecutively are grouped into a single undo action. A `Node` is created and pushed onto the undo stack when any of the following occur:

- The cursor moves.
- The Enter key is pressed.
- The Backspace key is pressed.

At that point, the accumulated text is stored in a `Node`, and the insertion `StringBuilder` is reset.

Text Deletion

Similarly, consecutive deletions are grouped together. A `Node` is created when:

- The cursor moves.

- The Enter key is pressed.
- New text is entered.

The deleted text is then stored in a `Node`, and the deletion `StringBuilder` is reset.

Line Operations

Creating a new line or deleting a line is always treated as a separate undo action, with its own `Node`.

Immediate Undo Handling

Because insertion and deletion operations are grouped and only committed when another action occurs, a user could type or delete text and immediately press **Ctrl + Z**. To handle this case, I added helper functions within the `Input` class that force any pending edits to be committed before performing the undo operation. The associated `StringBuilder` is then reset.

Redo

Redo functionality is handled through a `RedoStack` object. Whenever the user performs an undo, the redo information is stored in a new `RedoStack` instance. One thing to mention is that my Text Editor only has one level or redo's.

Any time a new edit is made, the redo history becomes invalid, so `RedoStack` is set to `null`. Likewise, after performing a redo operation, the redo information is cleared by setting `RedoStack` to `null` again. This ensures that redo is only available immediately after an undo and prevents invalid redo operations after new edits are introduced.

Rendering

For my text editor, I used Java Swing for the graphics and UI. While Swing is not as performant as a dedicated graphics API such as OpenGL, I implemented several optimizations and design decisions to improve rendering speed and overall efficiency.

Text Rendering

Initially, text rendering was performed by looping through an `ArrayList<GapBuffer>` and drawing each character individually. While this worked for smaller files, it became increasingly inefficient as file sizes grew. To address this, I implemented three main optimizations.

Rendering Only Visible Lines

The first optimization was limiting rendering to only the lines currently visible on the screen. Since off-screen lines cannot be seen, rendering them is unnecessary work. Two variables, `Y_start` and `Y_end`, determine the first and last visible lines that need to be drawn. This optimization is closely tied to the editor's vertical scrolling system, as the same values are used to determine which portion of the document is currently in view.

Reducing drawString() Calls

The second optimization focused on minimizing calls to `g2d.drawString()`. Originally, `drawString()` was called once for every character while iterating through the text. This resulted in a large number of rendering calls when displaying substantial amounts of text.

To improve performance, I introduced a `StringBuilder` that constructs an entire line of text before rendering it with a single `drawString()` call. After drawing the line, the `StringBuilder` is cleared and reused for the next line. This significantly reduces rendering overhead.

One challenge with this approach is that character spacing can vary slightly when Java renders a complete string. Even with monospaced fonts, Java's text rendering can apply minor horizontal adjustments, causing the cursor—which has a fixed width—to drift from its expected position as more text is added.

To maintain accurate cursor positioning, I calculate the cursor's x-coordinate by constructing a string containing all characters before the cursor and measuring its width using:

```
fm.stringWidth(sb.toString()) + OFFSET_X
```

This provides the exact pixel position needed to render the cursor correctly.

Skipping the Gap in the Gap Buffer

The third optimization takes advantage of the Gap Buffer data structure. Since the gap contains only null characters, iterating through it would waste processing time.

To avoid this, each line is rendered in two separate sections:

1. The text before the gap.
2. The text after the gap.

The first loop renders characters from the start of the line up to the beginning of the gap. If the cursor is positioned at the start of the line, this loop naturally performs no iterations.

The second loop renders characters from the end of the gap to the end of the line. Likewise, if the cursor is positioned at the end of the line, the loop performs no iterations.

By skipping the gap entirely, unnecessary processing is eliminated and rendering performance is improved.

Repaint Optimizations

Another significant optimization involved reducing the amount of screen area that needed to be repainted.

In Swing, calling `repaint()` without arguments schedules a repaint of the entire component. While functional, this can become expensive in a text editor where screen updates occur frequently. Instead, I made use of the overloaded `repaint(int x, int y, int width, int height)` method, which allows only a specific rectangular region of the screen to be refreshed.

This approach minimizes the amount of work required for each update and improves responsiveness.

Text Insertion and Deletion

When text is inserted or deleted, only the current line is repainted, beginning at the start of the line and extending to the edge of the screen.

Adding a New Line

When a new line is inserted, the editor repaints from the insertion point to the bottom of the visible screen. This is necessary because all subsequent lines shift downward and must be redrawn in their new positions.

Deleting a Line

Deleting a line follows the same principle. The editor repaints from the deleted line to the bottom of the screen because all following lines shift upward.

Cursor Movement

When the cursor moves, only two small regions are repainted:

- The cursor's previous position, to erase it.
- The cursor's new position, to draw it.

This avoids unnecessary updates to the rest of the document.

Button Interactions

The editor also includes **New**, **Open**, and **Save** buttons for file operations. When these buttons are hovered over or pressed, only the button's area is repainted rather than the entire window.

Scrolling

The only scenario that requires a full-screen repaint is scrolling. Whether scrolling occurs through explicit scroll actions, cursor movement, or text insertion that shifts the viewport, the visible portion of the document changes. As a result, the entire screen must be repainted to display the updated view correctly.